TÀI LIỆU THIẾT KẾ HƯỚNG ĐỐI TƯỢNG (MÔ HÌNH LỚP)

HỆ THỐNG QUẢN LÝ CHI TIÊU CÁ NHÂN CÓ TÍCH HỢP AI

Phiên bản 2.1 — As-built, schema chốt triển khai
Cập nhật theo mã nguồn ngày 02/09/2026

Nhóm 21 - Thành viên nhóm 2
1. Đào Minh Tâm (Trưởng nhóm)
2. Phạm Đức Nam

Thời gian thực hiện: 27/07/2026 – 27/09/2026 (9 tuần)

1. Mô hình lớp (Class Diagram)

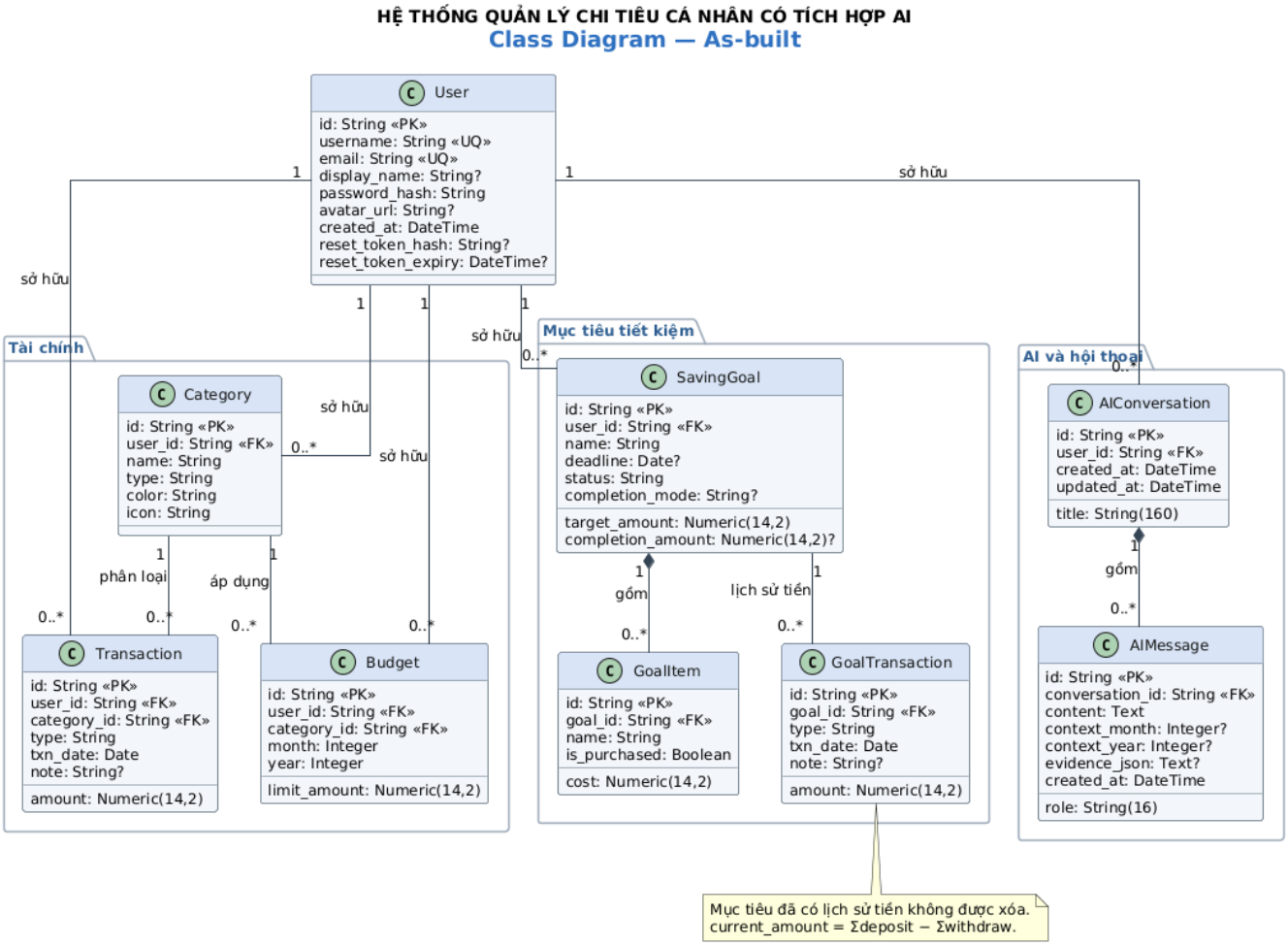
1.1. Giới thiệu mô hình lớp

Mô hình lớp mô tả cấu trúc tĩnh đang tồn tại trong mã nguồn, gồm các lớp SQLAlchemy, thuộc tính lưu trữ và quan hệ khóa ngoại. Bản cập nhật lấy app/models, migration 20260901_0001 và các router hiện hành làm chuẩn.

Hệ thống hiện có 9 lớp thực thể lưu trữ: User, Category, Transaction, Budget, SavingGoal, GoalItem, GoalTransaction, AIConversation và AIMessage. Các model là lớp dữ liệu; nghiệp vụ không được cài dưới dạng phương thức entity mà nằm chủ yếu ở Router và các hàm Core.

Điểm hiệu chỉnh: Bản cũ chỉ có 8 lớp, thiếu AIConversation/AIMessage, thiếu một số cột mới và mô tả AI như định hướng. Bản này phản ánh Assistant dùng Gemini đã được triển khai. Model/bảng AIReport không có use case đã được loại bỏ trước khi chốt schema triển khai.

1.2. Class Diagram tổng thể



Hình 1. Class Diagram as-built, đối chiếu với cấu trúc bảng hiện tại

1.3. Mô tả các nhóm lớp

Nhóm	Thành phần hiện tại	Chức năng
Entity/Model	9 lớp SQLAlchemy	Ánh xạ dữ liệu nghiệp vụ và lịch sử hội thoại.
Schema	user, category, transaction, budget, goal, dashboard, report, assistant	Kiểm tra và định dạng I/O bằng Pydantic.
Router	auth, categories, transactions, budgets, goals, dashboard, reports, assistant	HTTP controller và phần lớn điều phối nghiệp vụ.
Core	security, deps, finance, email, assistant_context, gemini, config	Xác thực, tính toán dùng chung và tích hợp ngoài.
Database	engine, SessionLocal, Base, get_db, Alembic	Kết nối, phiên ORM và migration.

Nhóm	Thành phần hiện tại	Chức năng
Presentation	React + TypeScript, PWA, Windows/Tauri	Giao diện và gọi API dùng chung.
External	Google Gemini, Resend	Tư vấn tài chính và gửi email đặt lại mật khẩu.

1.4. Quan hệ giữa các lớp

Quan hệ	Bội số	Ý nghĩa as-built
User – Category/Transaction/Budget/SavingGoal/AIConversation	1 : 0..*	Các bảng con có user_id; mọi truy vấn nghiệp vụ giới hạn theo user.
Category – Transaction	1 : 0..*	Transaction.category_id phân loại giao dịch.
Category – Budget	1 : 0..*	Budget.category_id áp dụng hạn mức cho danh mục.
SavingGoal – GoalItem	1 : 0..*	GoalItem thuộc mục tiêu; router xóa item trước khi xóa goal.
SavingGoal – GoalTransaction	1 : 0..*	Nhật ký nạp/rút; goal có lịch sử tiền bị chặn xóa.
AIConversation – AIMessage	1 : 0..*	Router xóa messages trước khi xóa conversation.

1.5. Bội số (Multiplicity)

Ký hiệu	Ý nghĩa
1	Chính xác một đối tượng.
0..1	Không có hoặc có một đối tượng.
0..*	Không có hoặc có nhiều đối tượng.
1..*	Có ít nhất một đối tượng.

Các model hiện khai báo ForeignKey nhưng không khai báo SQLAlchemy relationship/ondelete. Vì vậy, một số ý nghĩa vòng đời được thi hành trong Router thay vì cascade ở ORM/database.

1.6. Vai trò của từng lớp

Lớp	Vai trò
User	Tài khoản, hồ sơ, avatar và khôi phục mật khẩu.
Category	Danh mục income/expense với màu và icon.
Transaction	Khoản thu/chi cụ thể; nguồn dữ liệu tổng hợp.
Budget	Hạn mức theo danh mục và tháng/năm.
SavingGoal	Mục tiêu, trạng thái và kết quả hoàn thành.
GoalItem	Hạng mục con của mục tiêu.
GoalTransaction	Nhật ký deposit/withdraw và nguồn tính current_amount.
AIConversation	Luồng hội thoại AI của người dùng.
AIMessage	Câu hỏi/câu trả lời cùng kỳ và evidence snapshot.

1.7. Nguyên tắc thiết kế hướng đối tượng

- Entity mỏng: model tập trung vào ánh xạ dữ liệu, không gắn nhầm các endpoint thành phương thức entity.
- Phân tách theo module chức năng: mỗi Router/Schema/Model phục vụ một miền nghiệp vụ.
- Dependency Injection: Session và current_user được cấp qua Depends.
- Tái sử dụng có chọn lọc: finance, security, email, assistant_context và gemini được tách thành Core.
- Ranh giới riêng tư: Assistant chỉ nhận dữ liệu tổng hợp đã loại định danh và dữ liệu thô.

2. Đặc tả các lớp (Class Specification)

Tên thuộc tính dưới đây dùng snake_case để khớp trực tiếp với mã Python và schema database. Mục "Thao tác liên quan" chỉ rõ nơi cài đặt nghiệp vụ; đây không phải phương thức của lớp ORM.

2.1. Đặc tả lớp User

Ánh xạ bảng users; lưu danh tính, thông tin hồ sơ, mật khẩu băm và dữ liệu đặt lại mật khẩu.

Thuộc tính

Thuộc tính	Kiểu dữ liệu	Mô tả
id	String (UUID)	PK; sinh UUID
username	String	Bắt buộc; duy nhất; có index
email	String	Bắt buộc; duy nhất; có index
display_name	String?	Tên hiển thị tùy chọn
password_hash	String	Mật khẩu đã băm bằng bcrypt
avatar_url	String?	Đường dẫn ảnh đại diện
created_at	DateTime(timezone)	Thời điểm tạo
reset_token_hash	String?	SHA-256 của token đặt lại
reset_token_expiry	DateTime(timezone)?	Hạn hiệu lực token

Thao tác liên quan (Router/Core)

Thao tác	Nơi cài đặt	Trách nhiệm
register, login, me	AuthRouter	Đăng ký, đăng nhập và lấy người dùng hiện tại
update_profile	AuthRouter	Cập nhật display_name, username, email
upload_avatar, delete_avatar	AuthRouter + file storage	Quản lý avatar PNG/JPEG/WebP tối đa 2 MB
forgot_password, reset_password	AuthRouter + Security + Resend	Khôi phục mật khẩu bằng token dùng một lần

Quan hệ: 1:N với Category, Transaction, Budget, SavingGoal và AIConversation.

Quy tắc nghiệp vụ

- Username dài 3–50 ký tự; email hợp lệ; username và email duy nhất.
- Mật khẩu đầu vào tối thiểu 6 ký tự và chỉ lưu dạng bcrypt hash.
- JWT xác thực chứa user id; nghiệp vụ chính lọc dữ liệu theo current_user.id.
- Yêu cầu quên mật khẩu tối đa 3 lần/email/giờ trong bộ nhớ; token hết hạn sau 15 phút.
- Khi email thay đổi, reset token hiện có bị vô hiệu hóa.

2.2. Đặc tả lớp Category

Ánh xạ bảng categories; mô tả danh mục thu/chi riêng của từng người dùng.

Thuộc tính

Thuộc tính	Kiểu dữ liệu	Mô tả
id	String (UUID)	PK
user_id	String	FK → users.id
name	String	Tên danh mục
type	String	income hoặc expense
color	String	Màu hiển thị; mặc định #D9A441
icon	String	Khóa icon; mặc định circle-dollar-sign

Thao tác liên quan (Router/Core)

Thao tác	Nơi cài đặt	Trách nhiệm
list_categories	CategoriesRouter	Liệt kê theo người dùng
create_category	CategoriesRouter	Tạo danh mục
update_category	CategoriesRouter	Cập nhật tên, loại, màu, icon
delete_category	CategoriesRouter	Xóa khi không có giao dịch tham chiếu

Quan hệ: N:1 với User; 1:N với Transaction và Budget.

Quy tắc nghiệp vụ

- Cặp name + type không được trùng trong cùng tài khoản.

• Type chỉ nhận income hoặc expense; icon phải thuộc tập khóa được Schema cho phép.
• Router hiện chặn xóa khi có Transaction tham chiếu; khóa ngoại Budget vẫn phải được bảo toàn.

2.3. Đặc tả lớp Transaction

Ánh xạ bảng transactions; lưu khoản thu/chi và là nguồn dữ liệu cho Dashboard, Reports và Assistant.

Thuộc tính

Thuộc tính	Kiểu dữ liệu	Mô tả
id	String (UUID)	PK
user_id	String	FK → users.id
category_id	String	FK → categories.id
amount	Numeric(14,2)	Số tiền chính xác
type	String	Suy ra từ Category.type
txn_date	Date	Ngày giao dịch
note	String?	Ghi chú tùy chọn

Thao tác liên quan (Router/Core)

Thao tác	Nơi cài đặt	Trách nhiệm
list_transactions	TransactionsRouter	Lọc theo category, type, ngày và từ khóa
create_transaction	TransactionsRouter	Tạo giao dịch
update_transaction	TransactionsRouter	Cập nhật và đồng bộ type khi đổi category
delete_transaction	TransactionsRouter	Xóa giao dịch thuộc người dùng

Quan hệ: N:1 với User và Category.

Quy tắc nghiệp vụ

• Amount phải lớn hơn 0; ngày giao dịch không được ở tương lai.
• Category phải thuộc người dùng hiện tại.
• Client không quyết định type; backend lấy type từ Category.

2.4. Đặc tả lớp Budget

Ánh xạ bảng budgets; lưu hạn mức theo danh mục chi và kỳ tháng/năm.

Thuộc tính

Thuộc tính	Kiểu dữ liệu	Mô tả
id	String (UUID)	PK
user_id	String	FK → users.id
category_id	String	FK → categories.id
month	Integer	1–12
year	Integer	Năm áp dụng
limit_amount	Numeric(14,2)	Hạn mức > 0

Thao tác liên quan (Router/Core)

Thao tác	Nơi cài đặt	Trách nhiệm
list_budgets	BudgetsRouter	Liệt kê và tính spent/is_over
create_budget	BudgetsRouter	Tạo ngân sách
update_budget	BudgetsRouter	Cập nhật hạn mức
delete_budget	BudgetsRouter	Xóa ngân sách
compute_spent	BudgetsRouter	SUM giao dịch chi theo category và kỳ

Quan hệ: N:1 với User và Category.

Quy tắc nghiệp vụ

• Chỉ Category loại expense của đúng người dùng được dùng để tạo Budget.
• Một category chỉ có một Budget trong cùng month/year theo kiểm tra nghiệp vụ.
• spent và is_over là dữ liệu tính động, không phải cột lưu trữ.

2.5. Đặc tả lớp SavingGoal

Ánh xạ bảng saving_goals; lưu mục tiêu và kết quả hoàn thành, không lưu current_amount.

Thuộc tính

Thuộc tính	Kiểu dữ liệu	Mô tả
id	String (UUID)	PK
user_id	String	FK → users.id
name	String	Tên mục tiêu
target_amount	Numeric(14,2)	Mức cần đạt
deadline	Date?	Hạn tùy chọn
status	String	active hoặc completed
completion_amount	Numeric(14,2)?	Số tiền tại lúc hoàn thành
completion_mode	String?	keep, release hoặc spend

Thao tác liên quan (Router/Core)

Thao tác	Nơi cài đặt	Trách nhiệm
list/create/update/delete_goal	GoalsRouter	Quản lý mục tiêu
to_goal_out	GoalsRouter	Ghép item và current_amount tính động
complete_goal	GoalsRouter	Hoàn thành theo keep/release/spend

Quan hệ: N:1 với User; 1:N với GoalItem và GoalTransaction.

Quy tắc nghiệp vụ

- Tên không rỗng, tối đa 100 ký tự; target_amount > 0.
- current_amount = tổng deposit − tổng withdraw.
- Chỉ hoàn thành khi current_amount ≥ target_amount.
- Không được xóa mục tiêu đã có lịch sử tiền; GoalItem được xóa thủ công khi xóa mục tiêu hợp lệ.

2.6. Đặc tả lớp GoalItem

Ánh xạ bảng goal_items; chia mục tiêu thành các hạng mục nhỏ.

Thuộc tính

Thuộc tính	Kiểu dữ liệu	Mô tả
id	String (UUID)	PK
goal_id	String	FK → saving_goals.id
name	String	Tên hạng mục
cost	Numeric(14,2)	Chi phí > 0
is_purchased	Boolean	Mặc định false

Thao tác liên quan (Router/Core)

Thao tác	Nơi cài đặt	Trách nhiệm
add_item	GoalsRouter	Thêm hạng mục
update_item	GoalsRouter	Sửa tên, cost, trạng thái mua
delete_item	GoalsRouter	Xóa hạng mục

Quan hệ: N:1 với SavingGoal; vòng đời được điều phối qua GoalsRouter.

Quy tắc nghiệp vụ

- Tên không được rỗng; cost phải lớn hơn 0.
- Mọi thao tác đều kiểm tra SavingGoal thuộc current_user.

2.7. Đặc tả lớp GoalTransaction

Ánh xạ bảng goal_transactions; lưu nhật ký nạp/rút và là cơ sở tính số tiền hiện có.

Thuộc tính

Thuộc tính	Kiểu dữ liệu	Mô tả
id	String (UUID)	PK
goal_id	String	FK → saving_goals.id
amount	Numeric(14,2)	Số tiền > 0
type	String	deposit hoặc withdraw
txn_date	Date	Ngày ghi nhận
note	String?	Ghi chú tùy chọn

Thao tác liên quan (Router/Core)

Thao tác	Nơi cài đặt	Trách nhiệm
list_goal_transactions	GoalsRouter	Lịch sử giảm dần theo ngày

Thao tác	Nơi cài đặt	Trách nhiệm
deposit	GoalsRouter + Finance	Nạp khi goal active và đủ số dư
withdraw	GoalsRouter	Rút không vượt current_amount
compute_current	GoalsRouter	SUM deposit − SUM withdraw

Quan hệ: N:1 với SavingGoal.

Quy tắc nghiệp vụ

- Amount phải lớn hơn 0.
- Nạp không vượt số dư khả dụng; rút không vượt số đã tiết kiệm.
- Nhật ký tiền được bảo toàn: mục tiêu có bản ghi này không được xóa.

2.8. Đặc tả lớp AIConversation

Ánh xạ bảng ai_conversations; đại diện một luồng hội thoại của người dùng với trợ lý.

Thuộc tính

Thuộc tính	Kiểu dữ liệu	Mô tả
id	String (UUID)	PK
user_id	String	FK → users.id; có index
title	String(160)	Mặc định Cuộc trò chuyện mới
created_at	DateTime(timezone)	Thời điểm tạo
updated_at	DateTime(timezone)	Cập nhật khi có câu trả lời

Thao tác liên quan (Router/Core)

Thao tác	Nơi cài đặt	Trách nhiệm
list_conversations	AssistantRouter	Danh sách theo updated_at giảm dần
get_conversation	AssistantRouter	Lấy hội thoại và messages
delete_conversation	AssistantRouter	Xóa messages rồi xóa conversation
ask_assistant	AssistantRouter	Tạo conversation mới khi chưa có id

Quan hệ: N:1 với User; 1:N composition với AIMessage ở tầng nghiệp vụ.

Quy tắc nghiệp vụ

- Mọi thao tác kiểm tra conversation_id cùng user_id.
- Tiêu đề hội thoại mới lấy tối đa 72 ký tự đầu của câu hỏi.

2.9. Đặc tả lớp AIMessage

Ánh xạ bảng ai_messages; lưu câu hỏi, câu trả lời và bằng chứng tổng hợp tại thời điểm trả lời.

Thuộc tính

Thuộc tính	Kiểu dữ liệu	Mô tả
id	String (UUID)	PK
conversation_id	String	FK → ai_conversations.id; có index
role	String(16)	user hoặc assistant
content	Text	Nội dung thông điệp
context_month	Integer?	Tháng của evidence
context_year	Integer?	Năm của evidence
evidence_json	Text?	Snapshot dữ liệu tổng hợp
created_at	DateTime(timezone)	Thời điểm tạo

Thao tác liên quan (Router/Core)

Thao tác	Nơi cài đặt	Trách nhiệm
_message_out	AssistantRouter	Giải mã evidence_json cho response
ask_assistant	AssistantRouter	Lưu cặp user/assistant sau khi Gemini thành công

Quan hệ: N:1 với AIConversation.

Quy tắc nghiệp vụ

- Role đầu ra chỉ là user hoặc assistant.
- Không lưu message mới khi Gemini lỗi; phiên DB được rollback.
- Evidence gắn kỳ và số liệu tổng hợp để có thể giải thích câu trả lời.

2.10. Đặc tả module Dashboard

Dashboard không phải entity. DashboardRouter tổng hợp dữ liệu trực tiếp từ Transaction, Category, Budget, SavingGoal và GoalTransaction, đồng thời gọi compute_available_balance.

Đầu ra	Nguồn/tính toán
summary	Thu, chi, net và available_balance theo kỳ.
spending_by_category	SUM expense theo Category.
trend	Thu/chi của 3 tháng gần nhất.
budgets	Hạn mức, spent, remaining, usage %, status.
goals	Tối đa 3 goal, current_amount và progress %.
recent_transactions	Tối đa 5 giao dịch gần nhất trong kỳ.

2.11. Đặc tả module Reports

ReportsRouter tạo báo cáo so sánh kỳ được chọn với tháng liền trước. Kết quả được tính xác định từ Transaction/Category và trả trực tiếp; không gọi Gemini và không lưu một bảng báo cáo riêng.

Kết quả	Nội dung
Summary	Income, expense, net của hai kỳ; difference và change percentage.
Category comparison	Amount, share, transaction_count và chênh lệch theo danh mục.
has_data	Cho phép giao diện chọn biểu đồ hoặc empty state.

2.12. Đặc tả module Assistant

AssistantRouter cung cấp context tổng hợp, quản lý lịch sử hội thoại và gửi câu hỏi tới Gemini. AssistantContext tạo evidence cho kỳ hiện tại/tháng trước; GeminiService chỉ sinh tư vấn read-only.

API	Trách nhiệm
GET /assistant/context	Evidence, trạng thái có dữ liệu, ghi chú riêng tư và model.
GET /assistant/conversations	Danh sách hội thoại theo thời gian cập nhật.
GET /assistant/conversations/{id}	Chi tiết hội thoại và messages.
DELETE /assistant/conversations/{id}	Xóa messages rồi xóa conversation.
POST /assistant/messages	Gọi Gemini; lưu cặp message và evidence nếu thành công.

3. Thiết kế Package

3.1. Mục đích

Package được mô tả theo đúng cấu trúc thư mục hiện hành, giúp truy vết từ giao diện tới API, nghiệp vụ dùng chung và dữ liệu.

3.2. Các Package của hệ thống

STT	Package	Thành phần chính	Chức năng
1	frontend/src	pages, components, auth, lib	React UI, quản lý phiên và gọi API.
2	app/routers	8 router chức năng	HTTP controller và điều phối nghiệp vụ.
3	app/schemas	8 nhóm Pydantic schema	Validate/serialize request-response.
4	app/models	9 lớp SQLAlchemy	Ánh xạ bảng.
5	app/core	security, deps, finance, email, assistant_context, gemini, config	Logic dùng chung và tích hợp.
6	app/database.py	Engine, SessionLocal, Base, get_db	Truy cập database.
7	alembic	migration 20260901_0001	Quản lý schema production.
8	uploads	avatars	Lưu tệp ảnh đại diện.

3.3. Mô tả các Package

Presentation hỗ trợ web responsive/PWA và Windows/Tauri, cùng kết nối tới FastAPI. Router sử dụng Schema, Model và Core; Core có thể truy vấn Model qua Session hoặc gọi dịch vụ ngoài. Không có package Repository tổng quát trong mã nguồn hiện tại.

3.4. Quan hệ giữa các Package

Luồng chính: frontend → Router → Schema/Core/Model → Database. Riêng quên mật khẩu gọi Resend; AssistantContext đọc dữ liệu tổng hợp và GeminiService gọi Google Gemini.

3.5. Lợi ích và giới hạn

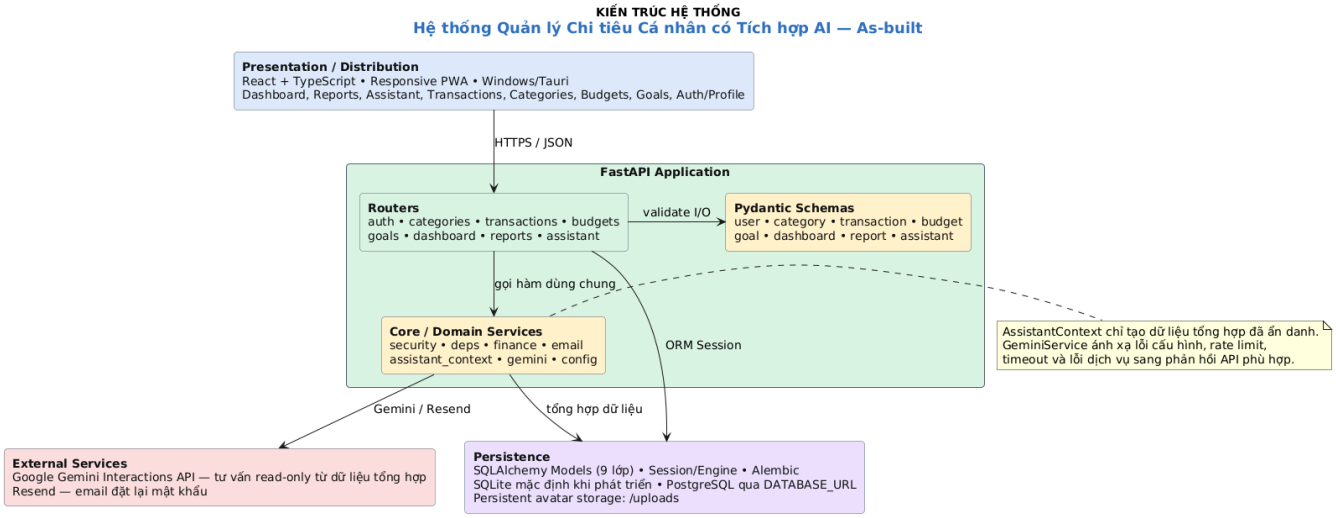
- Cấu trúc theo chức năng dễ tìm và phù hợp quy mô hiện tại.
- Schema tách validation khỏi ORM; Core giảm lặp cho nghiệp vụ dùng chung.
- Router vẫn chứa nhiều điều phối; khi nghiệp vụ tăng có thể tách service/repository có kiểm soát.

4. Thiết kế Service Layer

4.1. Giới thiệu

Hệ thống dùng kiến trúc router-centric có Core services, không phải mô hình hoàn toàn không có Service Layer. CRUD và điều phối theo request nằm trong Router; logic tái sử dụng hoặc tích hợp nằm trong app/core.

4.2. Kiến trúc xử lý nghiệp vụ



Hình 2. Kiến trúc as-built của hệ thống

4.3. Các nhóm nghiệp vụ

Nhóm	Router/Core	Chức năng hiện hành
Auth/Profile	auth + security + email	Tài khoản, JWT, hồ sơ, avatar, reset password.
Category	categories	CRUD danh mục.
Transaction	transactions	CRUD và lọc giao dịch.
Budget	budgets	CRUD, spent và is_over.
SavingGoal	goals + finance	Goal/item, nạp/rút, hoàn thành.
Dashboard	dashboard + finance	Tổng hợp màn hình tổng quan.
Reports	reports	So sánh tháng hiện tại/tháng trước; không AI.
Assistant	assistant + assistant_context + gemini	Evidence, Gemini, lịch sử hội thoại.

4.4. Core services dùng chung

Module	Trách nhiệm
security.py	Băm/xác minh mật khẩu; tạo/giải mã JWT.
deps.py	Lấy current_user từ bearer token.
finance.py	Tính available_balance từ thu, chi và dòng tiền mục tiêu.
email.py	Gửi email đặt lại mật khẩu qua Resend.
assistant_context.py	Tạo evidence tổng hợp, forecast và nhãn danh mục an toàn.
gemini.py	Tạo prompt, gọi Interactions API, retry và phân loại lỗi.
config.py	Cấu hình môi trường, CORS, trusted hosts và API docs.

4.5. Luồng xử lý — ví dụ thêm giao dịch

1. TransactionsPage gửi category_id, amount, txn_date và note.
2. TransactionCreate kiểm tra amount > 0 và ngày không ở tương lai.
3. TransactionsRouter xác thực Category thuộc current_user.
4. Router suy ra Transaction.type từ Category.type.
5. SQLAlchemy Session INSERT và COMMIT Transaction.
6. API trả TransactionOut; Dashboard/Reports sẽ tổng hợp từ dữ liệu mới ở lần đọc tiếp theo.

4.6. Ưu điểm và điểm cần lưu ý

• Luồng request ngắn và dễ theo dõi cho CRUD.

• Core services đã tách các phần phức tạp/được tái sử dụng.

- Một số quy tắc toàn vẹn mới ở mức Router, chưa có UniqueConstraint/cascade tương ứng trong migration.
- Rate limit reset password lưu trong bộ nhớ, phù hợp đồ án nhưng không chia sẻ giữa nhiều process.

4.7. Kết luận

Kiến trúc hiện tại phù hợp với quy mô dự án. Nếu mở rộng, nên ưu tiên tách service theo use case có nhiều transaction boundary, thay vì thêm tầng trung gian cho mọi CRUD.

5. Thiết kế Database Layer

5.1. Mục đích

Database Layer dùng SQLAlchemy ORM để quản lý kết nối, Session và ánh xạ 9 bảng. get_db cấp một Session cho mỗi request và luôn đóng Session trong khối finally.

5.2. Thành phần

Thành phần	Chức năng as-built
engine	Tạo từ DATABASE_URL, bật pool_pre_ping.
SessionLocal	autocommit=False, autoflush=False.
Base	declarative_base cho các model.
get_db()	Cấp và đóng Session qua Depends.
Alembic	Migration production 20260901_0001.
schema_migrations	Bổ sung cột tương thích khi AUTO_CREATE_SCHEMA ở development.

5.3. Cấu hình

- Mặc định phát triển: sqlite:///./expense.db và check_same_thread=False.
- PostgreSQL được hỗ trợ qua DATABASE_URL; tiền tố postgresql:// được đổi sang postgresql+psycopg://.
- AUTO_CREATE_SCHEMA mặc định bật ở development và tắt ở production.
- API có /api/health và /api/ready; readiness thực hiện SELECT 1.

5.4. Quy trình ghi dữ liệu

1. Schema kiểm tra kiểu và ràng buộc đầu vào.
2. Router xác thực quyền sở hữu và quy tắc nghiệp vụ.
3. Tạo/cập nhật/xóa model trong Session.
4. COMMIT; khi cần thì refresh đối tượng.
5. Khi tích hợp ngoài lỗi trong Assistant, ROLLBACK trước khi trả lỗi.
6. get_db đóng Session khi request kết thúc.

5.5. Ràng buộc và tính toàn vẹn

- Username/email có unique index trong migration.
- Các quan hệ dùng ForeignKey; chưa khai báo ondelete/cascade ở schema.
- Một số ràng buộc tổ hợp như Category(name,type,user) và Budget(category,month,year,user) được kiểm tra ở Router, chưa có UniqueConstraint.
- Numeric(14,2) được dùng cho tiền, tránh Float.
- Avatar được lưu ở file storage; DB chỉ lưu avatar_url.

6. Thiết kế AI Module

6.1. Trạng thái triển khai

AI Module không còn là định hướng. AssistantPage, AssistantRouter, AssistantContext, GeminiService, AIConversation và AIMessage đã có trong mã nguồn. Model mặc định là gemini-3.7-flash và có thể đổi bằng GEMINI_MODEL.

6.2. Dữ liệu đầu vào an toàn

AssistantContext tổng hợp kỳ hiện tại và tháng liền trước: thu, chi, net, số giao dịch, số dư khả dụng, tối đa 7 nhóm chi tiêu theo nhãn icon an toàn, ngân sách, mục tiêu và forecast.

- Không gửi username, email, internal id, ghi chú giao dịch, giao dịch thô hoặc tên mục tiêu.
- Không gửi tên danh mục tùy biến; ánh xạ icon sang nhãn an toàn.
- Câu hỏi và tối đa 10 thông điệp lịch sử dạng văn bản được dùng để tạo prompt.

6.3. Chức năng AI hiện hành

Chức năng	Trạng thái
Xem phạm vi dữ liệu/evidence	Đã triển khai
Hỏi đáp tư vấn tài chính	Đã triển khai qua Gemini
Dự báo chi tiêu cuối tháng	Đã triển khai trong evidence; confidence low/medium
Lịch sử hội thoại	Đã triển khai
Liên kết về Reports/Transactions	Được giao diện sử dụng để kiểm chứng dữ liệu
AI tự sửa giao dịch/ngân sách	Không hỗ trợ; read-only
Báo cáo so sánh tháng	Đã triển khai nhưng là deterministic, không dùng Gemini

6.4. Luồng xử lý AI

1. Người dùng chọn kỳ và nhập câu hỏi.
2. AssistantRouter kiểm tra hoặc tạo AIConversation.
3. AssistantContext truy vấn và ẩn danh dữ liệu tổng hợp.
4. GeminiService tạo system instruction, context JSON và lịch sử.
5. Gọi Google Gemini Interactions API với retry có giới hạn.
6. Nếu thành công, lưu cặp AIMessage và evidence_json; nếu lỗi, rollback.
7. Trả câu trả lời, bằng chứng, độ tin cậy và nguồn điều hướng cho giao diện.

6.5. Kiểm soát lỗi và an toàn

Tình huống	Phản hồi API
Chưa cấu hình API key	503
Rate limit	429
Timeout	504
Gemini không sẵn sàng	503
Lỗi dịch vụ/response không hợp lệ	502

6.6. Chiến lược lưu trữ kết quả AI

As-built: Câu hỏi, câu trả lời, kỳ phân tích và evidence snapshot được lưu trực tiếp trong AIConversation/AIMessage. Hệ thống không duy trì bảng báo cáo AI riêng.

7. Áp dụng các nguyên tắc thiết kế hướng đối tượng

7.1. Đánh giá SOLID theo mức áp dụng thực tế

Nguyên tắc	Mức áp dụng	Bằng chứng/giới hạn
SRP	Một phần	Module chia theo miền; một số Router vẫn vừa kiểm tra, truy vấn và điều phối.
OCP	Một phần	Thêm router/schema/core theo module; chưa có abstraction/plugin cho mọi nghiệp vụ.
LSP	Không phải trọng tâm	Không có hệ phân cấp kế thừa nghiệp vụ đáng kể ngoài BaseModel/Base ORM.
ISP	Phù hợp theo API	Schema request/response nhỏ theo từng use case.
DIP/DI	Có	FastAPI Depends cấp Session và current_user; tích hợp ngoài vẫn gọi qua module cụ thể.

7.2. Kiến trúc tổng thể

Kiến trúc thực tế là Presentation → Router → Schema/Core/Model → Database, kèm External Services. Bản Windows/Tauri và Web/PWA dùng chung backend FastAPI.

7.3. Dependency Injection

get_db và get_current_user được tiêm qua Depends vào endpoint. Cách này tách việc tạo Session/xác thực khỏi logic use case và hỗ trợ kiểm thử bằng dependency override.

7.4. Định hướng mở rộng phù hợp

- Bổ sung DB UniqueConstraint cho các ràng buộc tổ hợp quan trọng.
- Làm rõ cascade/ondelete hoặc tiếp tục điều phối vòng đời nhất quán trong service.
- Tách service cho Goals/Assistant nếu transaction boundary tiếp tục phức tạp.
- Thay rate limiter in-memory bằng shared store nếu chạy nhiều process.
- Nếu sau này có báo cáo AI định kỳ, thiết kế migration mới từ use case thực tế thay vì giữ bảng dự phòng.

8. Kết luận

Tài liệu đã được đồng bộ với dự án as-built ngày 02/09/2026. Hệ thống có 9 lớp lưu trữ, 8 router nghiệp vụ và kiến trúc router-centric kết hợp Core services. Dashboard và Reports là module tổng hợp không có entity riêng; Reports hoạt động xác định, còn Assistant dùng Gemini trên dữ liệu tổng hợp đã ẩn danh.

Thiết kế hiện tại phù hợp quy mô đồ án và đã hỗ trợ web/PWA/Windows dùng chung API. Các điểm cần theo dõi khi mở rộng là ràng buộc toàn vẹn ở database, vòng đời quan hệ, rate limiting phân tán và chiến lược migration sau khi schema production được chốt.